

Justificativa de Escolha dos Bancos e Armazenamentos

Projeto: Conexao Solidaria - Hackathon 9NETT

1. Contexto

A plataforma Conexao Solidaria foi planejada para atender uma ONG que precisa gerenciar doadores, campanhas e doacoes de forma digital, escalavel e observavel. A arquitetura adotada utiliza uma API de campanhas e usuarios, um Worker de processamento de doacoes, RabbitMQ para comunicacao assíncrona e SQL Server para persistencia relacional.

2. Banco Principal - SQL Server

- O SQL Server foi escolhido como banco de dados principal por possuir forte compatibilidade com o ecossistema .NET e Entity Framework Core.
- O dominio da aplicacao e naturalmente relacional: usuarios, campanhas e doacoes possuem chaves, relacionamentos e necessidade de consistencia transacional.
- As operacoes de cadastro, autenticao, consulta de campanhas e registro de doacoes exigem integridade dos dados, indices unicos e confiabilidade.
- O SQL Server permite criar restricoes, indices e relacionamentos adequados para garantir email unico, CPF unico e associacao correta entre doacoes e campanhas.
- Tambem e simples executar SQL Server localmente com Docker, o que facilita a correcao dos professores e a execucao em ambiente Kubernetes.

3. Dados Persistidos no SQL Server

Entidade	Finalidade
Usuarios	Armazena doadores e gestores, incluindo email, CPF, senha com hash e role.
Campanhas	Armazena titulo, descricao, periodo, meta financeira, status e total arrecadado.
Doacoes	Armazena a intencao de doacao enviada pelo doador autenticado.
StoredEvents	Opcionalmente registra eventos de dominio para auditoria e rastreabilidade.

4. Broker de Mensageria - RabbitMQ

Embora RabbitMQ nao seja um banco de dados, ele e um componente de armazenamento temporario de mensagens e foi escolhido para garantir desacoplamento entre a API e o Worker. Quando uma doacao e criada, a API publica um evento na fila. O Worker consome essa mensagem e atualiza o total arrecadado da campanha.

- Evita que a API atualize diretamente o valor arrecadado, cumprindo o requisito de comunicacao assíncrona.
- Permite que a API responda rapidamente ao usuario com status 202 Accepted.
- Garante maior escalabilidade, pois o Worker pode ser replicado conforme o volume de mensagens.

- Possui painel administrativo simples, facilitando a demonstração da fila durante o vídeo.

5. Observabilidade - Prometheus/Grafana

Prometheus e Grafana foram utilizados para coleta e visualização de métricas. Prometheus coleta dados expostos pelos endpoints /metrics, enquanto Grafana apresenta dashboards com informações como consumo de CPU, memória, quantidade de pods e requisições HTTP.

6. Conclusão

A combinação SQL Server + RabbitMQ atende aos requisitos funcionais e técnicos do projeto. O SQL Server oferece consistência e integridade para os dados principais, enquanto o RabbitMQ permite processamento assíncrono e desacoplado das doações. Essa arquitetura é adequada para um MVP robusto, preparado para crescimento e alinhado aos requisitos de microsserviços, mensageria, Kubernetes, CI/CD e observabilidade.